

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA0302	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	T. Bhanu Pavan Varma	Reg. No.:	192425380
Section / Batch:	AI&ML(2024-2028)	Date:	27/07/2026

Code of Conduct

I T. Bhanu Pavan Varma (192425380) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. Problem Statement:

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:

Email Address

Password

Mobile Number

using Regular Expressions.

Validation Rules

Email

Must start with an alphabet.

May contain letters, digits, '.', '_' before '@'.

Domain should contain only alphabets.

Valid extensions: .com, .org, .edu, .net, .in

Password

Minimum 8 characters

At least one uppercase letter

At least one lowercase letter

At least one digit

At least one special character (@, #, \$, %, &, !)

Mobile Number

Must contain exactly 10 digits.

First digit should be between 6–9.

Expected Output

Display:

Valid Email / Invalid Email

Strong Password / Weak Password

Valid Mobile Number / Invalid Mobile Number

Code:

```
import re

# Email validation
def validate_email(email):
    pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'
    return re.match(pattern, email) is not None

# Password validation
def validate_password(password):
    pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$$%&!]).{8,}$'
    return re.match(pattern, password) is not None

# Mobile number validation
def validate_mobile(mobile):
    pattern = r'^[6-9]\d{9}$'
    return re.match(pattern, mobile) is not None

# Get input from user
email = input("Enter Email Address: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")

# Display results
if validate_email(email):
    print("Valid Email")
else:
    print("Invalid Email")

if validate_password(password):
    print("Strong Password")
else:
    print("Weak Password")

if validate_mobile(mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

Sample Input:

Enter Email Address: student123@gmail.com

Enter Password: Student@123

Enter Mobile Number: 9876543210

Output:

Invalid Email

Weak Password

Section 2: Design a Finite State Automata (FSA) Simulator

1. Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
 - States
 - Input alphabet
 - Transition table
 - Initial state
 - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:
Strings ending with "ab"

Input
abaab

Output
Transition Path:
 $q_0 \rightarrow q_1 \rightarrow q_0 \rightarrow q_1 \rightarrow q_2$
Accepted

Code :

```
# DFA Simulator

# Accept DFA details
states = input("Enter states (comma separated): ").split(",")
alphabet = input("Enter input alphabet (comma separated): ").split(",")

states = [s.strip() for s in states]
alphabet = [a.strip() for a in alphabet]

# Transition table
transition = {}

print("\nEnter transition table:")

for state in states:
    transition[state] = {}

    for symbol in alphabet:
        next_state = input(
            f"Transition( {state}, {symbol} ) = "
        ).strip()

        transition[state][symbol] = next_state

# Initial state
initial_state = input("\nEnter initial state: ").strip()

# Final states
final_states = input(
    "Enter final state(s) (comma separated): "
).split(",")

final_states = [s.strip() for s in final_states]

# Number of input strings
n = int(input("\nEnter number of input strings: "))

# Process each string
for i in range(n):
    string = input(f"\nEnter input string {i + 1}: ").strip()

    current_state = initial_state
    path = [current_state]

    valid = True

    for symbol in string:
        if symbol not in alphabet:
```

Section 3: Smart Pattern Matching Engine using Regular Expressions

2. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:

Word Search
Prefix Search
Suffix Search
Date Search
Phone Number Search
Hashtag Search
Mention (@username) Search

Example Input

Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

User Menu

- 1.Search Date
- 2.Search Phone Number
- 3.Search Hashtag
- 4.Search Mention
- 5.Search Prefix
- 6.Search Suffix

Expected Output

Display all matching patterns based on user selection.

Code:

```
import re

# Sample text
text = """
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing
Python programming is useful for NLP applications.
"""

# Search Date
def search_date(text):
    pattern = r'\b\d{2}/\d{2}/\d{4}\b'
    return re.findall(pattern, text)
```

Search Phone Number

```
def search_phone(text):  
    pattern = r'\b[6-9]\d{9}\b'  
    return re.findall(pattern, text)
```

Search Hashtag

```
def search_hashtag(text):  
    pattern = r'#[A-Za-z0-9_]+'  
    return re.findall(pattern, text)
```

Search Mention

```
def search_mention(text):  
    pattern = r'@[A-Za-z0-9_]+'  
    return re.findall(pattern, text)
```

Prefix Search

```
def search_prefix(text, prefix):  
    pattern = r'\b' + re.escape(prefix) + r'\w*'  
    return re.findall(pattern, text, re.IGNORECASE)
```

Suffix Search

```
def search_suffix(text, suffix):  
    pattern = r'\b\w*' + re.escape(suffix) + r'\b'  
    return re.findall(pattern, text, re.IGNORECASE)
```

Word Search

```
def search_word(text, word):  
    pattern = r'\b' + re.escape(word) + r'\b'  
    return re.findall(pattern, text, re.IGNORECASE)
```

Display text

```
print("----- TEXT -----")  
print(text)  
while True:  
    print("\n----- SMART PATTERN MATCHING ENGINE -----")  
    print("1. Search Date")  
    print("2. Search Phone Number")  
    print("3. Search Hashtag")
```

```
print("4. Search Mention")
print("5. Search Prefix")
print("6. Search Suffix")
print("7. Search Word")
print("8. Exit")
```

```
choice = input("\nEnter your choice: ")
```

```
if choice == "1":
```

```
    result = search_date(text)
    print("\nMatching Dates:", result)
```

```
elif choice == "2":
```

```
    result = search_phone(text)
    print("\nMatching Phone Numbers:", result)
```

```
elif choice == "3":
```

```
    result = search_hashtag(text)
    print("\nMatching Hashtags:", result)
```

```
elif choice == "4":
```

```
    result = search_mention(text)
    print("\nMatching Mentions:", result)
```

```
elif choice == "5":
```

```
    prefix = input("Enter prefix: ")
    result = search_prefix(text, prefix)
    print("\nMatching Prefix Words:", result)
```

```
elif choice == "6":
```

```
    suffix = input("Enter suffix: ")
    result = search_suffix(text, suffix)
    print("\nMatching Suffix Words:", result)
```

```
elif choice == "7":
```

```
    word = input("Enter word: ")
    result = search_word(text, word)
    print("\nMatching Words:", result)
```

```
elif choice == "8":
```

```
    print("\nExiting program...")
```



```
break
else:
    print("\nInvalid choice. Please try again.")
```

Sample Output

----- SMART PATTERN MATCHING ENGINE -----

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit

Search Date

Enter your choice: 1

Matching Dates: ['12/09/2026']

Search Phone Number

Enter your choice: 2

Matching Phone Numbers: ['9876543210']

Search Hashtag

Enter your choice: 3

Matching Hashtags: ['#NLP']

Search Mention

Enter your choice: 4

Matching Mentions: ['@OpenAI']

Search Prefix

For example, search for words beginning with nat:

Enter your choice: 5

Enter prefix: nat

Matching Prefix Words: ['natural']

Search Suffix

For example, search for words ending with ing:

Enter your choice: 6

Enter suffix: ing

Matching Suffix Words: ['Meeting', 'processing', 'programming']

Search Word

Enter your choice: 7

Enter word: NLP

Matching Words: ['NLP']

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

